

CSC3066

Deep Learning

Anna Jurek-Loughrey



What's to come this week

- ❑ Neural Network models and text data
 - ❑ Bag of Words (BoW) methods for documents representation
 - ❑ Keras implementation of BoW methods



Working with text data

- Why do we need to work with text data?



Working with text data

□ Why do we need to work with text data?

Information Retrieval

Doc A



Doc 1

Doc 2

Doc 3

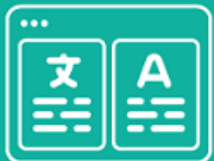
Sentiment Analysis



Information Extraction



Machine Translation



Text Processing

Question Answering



Human: When was Apollo sent to space?

Machine: First flight - AS-201, February 26, 1966



Neural Networks and text data

- ☐ Artificial Neural Networks don't take raw text as input , they only work with numerical values.
- ☐ To apply ANNs with text data we need to convert text into a vector.
- ☐ In other words, every word/document needs to have a numeric representation in order to be applied with ANNs.

Neural Network:

Bag of Words



One-hot encoded vectors: words representation

- ❑ One-hot encoded words:
 - Consider all possible words in a vocabulary.
 - Assign each word with a unique number.
 - Define a vector with the length of the vocabulary.
 - The value in each position in the vector refers to a single word from the vocabulary.
- ❑ For example, if the vocabulary of all words you will work with is : ['man', 'woman', 'red', 'blue', 'green'], then your vectors would have 5 dimensions.



One-hot encoded vectors: words representation

- Each word is represented as 1×5 vector:

Vocabulary:
'man', 'woman', 'red',
'blue', 'green'



	1	2	3	4	5
man	1	0	0	0	0
woman	0	1	0	0	0
red	0	0	1	0	0
blue	0	0	0	1	0
green	0	0	0	0	1

Each word gets a
 1×5 vector
representation



Bag of Words: document representation

Bag of Words (BoW):

- A bag-of-words is a representation of text that describes the occurrence of words within a document. It involves two things:
 - A vocabulary of known words
 - A measure of the presence of known words



Bag of Words: document representation

- BoW Step 1: Collecting your corpus:

- *"I am feeling very very happy"*

- *"I am not well today"*

- *"I want to be happy"*



Bag of Words: document representation

BoW Step 2: Creating the vocabulary

1. am
2. be
3. feeling
4. happy
5. i
6. not
7. to
8. today
9. very
10. want
11. well



Bag of Words: document representation

- ☐ BoW Step 2: Creating the vocabulary
- ☐ Before we create the vocabulary, we can apply some simple text cleaning techniques such as:
 - ☐ Ignoring cases
 - ☐ Ignoring stop word like 'a', 'the', ...
 - ☐ Fixing misspelled words
 - ☐ Reducing words to their stem (feel for feeling) using stemming algorithms
- ☐ Cleaning text can help to simplified the representation and reduce the size of the vocabulary



Bag of Words: document representation

- ☐ BoW Step 3: Creating document vectors
- ☐ The vocabulary has 11 words → fixed length document representation of 11, with one position in vector to score each word.
 - ☐ **Binary scoring**: binary scoring of presence or absence of words.
 - ☐ **Counts scoring**: counts the number of times each word appears in a document.
 - ☐ **Frequency scoring**: calculate how often each word appears within a document (as a ratio of words within each document).
 - ☐ **Term Frequency Inverse Document Frequency scoring (TFIDF)**



Bag of Words: document representation

□ BoW Step 3: Creating document vectors

1.am
2.be
3.feeling
4.happy
5.i
6.not
7.to
8.today
9.very
10.want
11.well

□ Binary scoring:

- "I am feeling very very happy": $[1\ 0\ 1\ 1\ 1\ 0\ 0\ 0\ 1\ 0\ 0]$
- "I am not well today": $[1\ 0\ 0\ 0\ 1\ 1\ 0\ 1\ 0\ 0\ 1]$
- "I want to be happy": *Try yourself*



Bag of Words: document representation

□ BoW Step 3: Creating document vectors

1.am
2.be
3.feeling
4.happy
5.i
6.not
7.to
8.today
9.very
10.want
11.well

□ Binary scoring:

- "I am feeling very very happy": $[1\ 0\ 1\ 1\ 1\ 0\ 0\ 0\ 1\ 0\ 0]$
- "I am not well today": $[1\ 0\ 0\ 0\ 1\ 1\ 0\ 1\ 0\ 0\ 1]$
- "I want to be happy": $[0\ 1\ 0\ 1\ 1\ 0\ 1\ 0\ 0\ 1\ 0]$



Bag of Words: document representation

- BoW Step 3: Creating document vectors

- Binary scoring:

Issue: ?



Bag of Words: document representation

☐ BoW Step 3: Creating document vectors

☐ Binary scoring:

Issue: Many times, the text semantics are determined by the number of appearances of some word. For example, if we read an article about “Vectors,” then the term “Vector” will appear many times. In that case, using just binary values in our text vectors will omit the true semantics of the text. The simplest solution is just to store the number of appearances of the word in the right position in the vector.



Bag of Words: document representation

□ BoW Step 3: Creating document vectors

1.am
2.be
3.feeling
4.happy
5.i
6.not
7.to
8.today
9.very
10.want
11.well

□ Counts scoring:

- *"I am feeling very very happy": [1 0 1 1 1 0 0 0 2 0 0]*
- *"I am not well today": [1 0 0 0 1 1 0 1 0 0 1]*
- *"I want to be happy": [0 1 0 1 1 0 1 0 0 1 0]*



Bag of Words: document representation

- BoW Step 3: Creating document vectors

- Counts scoring:

Issue: ?



Bag of Words: document representation

☐ BoW Step 3: Creating document vectors

☐ Counts scoring:

Issue: longer documents will have higher average count values than shorter documents, even though they might talk about the same topics.

To avoid these potential discrepancies it suffices to divide the number of occurrences of each word in a document by the total number of words in the document: these new features are called *tf* for Term Frequencies.



Bag of Words: document representation

□ BoW Step 3: Creating document vectors

1.am
2.be
3.feeling
4.happy
5.i
6.not
7.to
8.today
9.very
10.want
11.well

□ Frequency scoring:

- "I am feeling very very happy": $[0.17 \ 0 \ 0.17 \ 0.17 \ 0.17 \ 0 \ 0 \ 0 \ 0.33 \ 0 \ 0]$
- "I am not well today": $[0.2 \ 0 \ 0 \ 0 \ 0.2 \ 0.2 \ 0 \ 0.2 \ 0 \ 0 \ 0.2]$
- "I want to be happy": $[0 \ 0.2 \ 0 \ 0.2 \ 0.2 \ 0 \ 0.2 \ 0 \ 0 \ 0.2 \ 0]$



Bag of Words: document representation

- BoW Step 3: Creating document vectors

- Frequency scoring:

- Issue?



Bag of Words: document representation

☐ BoW Step 3: Creating document vectors

☐ Frequency scoring:

One issue with simple counts is that some words like “the” will appear many times and their large counts will not be very meaningful in the encoded vectors.

We would like to highlight words that are more interesting, e.g. frequent in a document but not across documents.



Bag of Words: document representation

- ❑ BoW Step 3: Creating document vectors (TFIDF)
 - ❑ **Term Frequency-Inverse Document Frequency**: statistic that intends to reflect how important a word w is to a document d in a collection of a documents $D = \{d_1, \dots, d_N\}$.
 - ❑ It is a product of two statistics:
 - ❑ **Term Frequency**: summarises how often a given word appears within a document
 - ❑ **Inverse Document Frequency**: downscales words that appear a lot across documents



Bag of Words: document representation

□ BoW Step 3: Creating document vectors (TFIDF)

□ Term Frequency (tf):

- Binary Frequency: $tf(w, d) = 1$ if w occurs in d and 0 otherwise
- Raw Count Frequency : $tf(w, d) = f_{w,d}$ (raw count of w in d)
- Frequency adjusted for doc length: $tf(w, d) = \frac{f_{w,d}}{\sum_{w' \in d} f_{w',d}}$
- Logarithmically scaled frequency: $tf(w, d) = \log(1 + f_{w,d})$



Bag of Words: document representation

❑ BoW Step 3: Creating document vectors (TFIDF)

❑ Inverse Document Frequency (*idf*)

- ❑ Measures how much information a word provides (i.e. if it is common or rare across all documents)

$$idf(w, D) = \log \frac{N}{|\{d \in D: w \in d\}|}$$

- ❑ $N = |D|$, total number of documents in the corpus
- ❑ $|\{d \in D: w \in d\}|$: number of documents where the word w occurs
- ❑ *idf*: logarithmically scaled inverse fraction of the documents that contain word w



Bag of Words: document representation

- BoW Step 3: Creating document vectors (TFIDF)

- Inverse Document Frequency (*idf*)

- $$idf(w, D) = \log \frac{N}{|\{d \in D: w \in d\}|} + 1$$

- By adding 1 to the *idf* term we prevent the words that occur in all documents in a training set from being entirely ignored.



Bag of Words: document representation

- BoW Step 3: Creating document vectors (TFIDF)
 - *tfidf* is calculated as:

$$tfidf(w, d, D) = tf(w, d) \times idf(w, D)$$

- High value of *tfidf* is obtained by high term frequency and low document frequency
- As a word appears in more documents, *tfidf* gets closer to 0



Bag of Words: document representation

☐ $tf(w, d) = \frac{f_{w,d}}{\sum_{w' \in d} f_{w'd}}$

$$idf(w, D) = \log \frac{N}{|\{d \in D: w \in d\}|}$$

☐ We have 1000 documents, 100 words each for simplicity

☐ Calculate tfidf for word 'the' in w_i knowing that it appears 10 times in w_i and it appears in 900 documents.

☐ Calculate tfidf for word 'python' in w_i knowing that it appears 10 times in w_i and it appears in 100 documents.

Exercise



Bag of Words: document representation

☐ $tf(w, d) = \frac{f_{w,d}}{\sum_{w' \in d} f_{w'd}}$

$$idf(w, D) = \log \frac{N}{|\{d \in D: w \in d\}|}$$

☐ We have 1000 documents, 100 words each for simplicity

☐ 'the': $(0.1 \times 0.046 = 0.0046)$

☐ 'python': $(0.1 \times 1 = 0.1)$

Bag of Words

Python



Bag of Words: document representation

- ❑ **Sklearn: CountVectorizer / TfidfVectorizer**
- ❑ Tokenise a collection of text documents, build a vocabulary and encode new documents using the vocabulary
 - `fit()`: learns vocabulary dictionary of all tokens in the raw documents.
 - `transform()`: transforms document to document-term matrix.
 - `fit_transform()`: learns vocabulary and returns document-term matrix

Python Programming Example



Bag of Words.ipynb



Bag of Words: document representation

- Limitations of Bag-of-Words



?



Bag of Words: document representation

❑ Limitations of Bag-of-Words

- **Sparsity:** The vocabulary requires careful design, most specifically in order to manage the size, which impacts the sparsity of the document representations. Sparse representations are harder to model both for computational reasons (space and time complexity) and also for information reasons, where the challenge is for the models to harness so little information in such a large representational space.
- **Meaning:** The model can't recognise words with the same meaning ("old" vs "used", "small" vs "tiny"), and much more.

Coming Next

Words Embeddings